LLM エージェントの履歴要約と終了制御による実行費用の評価

著者未設定

Evaluating Execution Cost of History Summarization and Termination Control for LLM Agents

Author details pending

所属・会員種別・連絡先：未設定

あらまし　LLM エージェントの履歴要約と終了制御を独立に評価した．要約生成を含む全モデル呼出しを計測し，反復要約と API 実行記録に基づく完了判定により，既知課題の成功を保ちながら総トークン数を削減できることを示した．

キーワード　LLM エージェント，ハーネス，履歴要約，終了制御，費用評価

1. はじめに

日常業務のデータ整理や事務処理の負担を軽減するため，大規模言語モデル（LLM）による作業支援が研究されている．文章の回答生成に加え，外部アプリケーションを操作する複数手順の実行も対象となる．例えば写真の整理や精算では，対象の検索，状態の変更，結果の確認を連続して行う必要がある．推論と行動を繰り返す ReAct[1]や，コードを行動として実行する CodeAct[2]は，このような対話的処理を支える技術である．

一方，実行が長くなると，過去の操作と観測を次の生成へ繰り返し入力する費用が増える．また，作業が終わった後も確認や生成を続ければ，成果を増やさずに資源を消費し得る．履歴を短くするだけでは必要な情報を失うおそれがあり，早く止めるだけでは未完了となり得るため，最終成功と実行全体の費用を同時に調べる必要がある．

本稿では，モデルを再学習せずに実行を管理するハーネスについて，履歴要約と終了判断の二つの制御を評価する．履歴要約では，要約を生成する呼出しの配置を比較する．終了判断では，実行中に取得できる API 応答の記録（以下，API 実行記録）に基づく制御を通常継続と比較する．両者を独立した実験として実行し，要約生成を含む全モデル呼出しの総トークン数と，課題の最終成功を調べる．

2. 関連研究

入力圧縮では，長文中の情報の位置が性能に影響すること[3]を踏まえ，LLMLingua[4]は予算制御とトークン選別，LLMLingua-2[5]は蒸留データによるトークン分類，LongLLMLingua[6]は長文の重要情報を残す圧縮を扱う．

長期実行では，ACON[7]は観測・履歴の圧縮方針，TokenPilot[8]は入力圧縮とキャッシュ継続性を扱う．Generative Agents[9]は経験の記憶・抽象化・検索，Reflexion[10]は言語的フィードバックを後続試行に利用する．本稿では，要約専用の呼出しで反復要約を行う方式と，行動生成と同時に要約を行う方式を比較し，要約生成も含む課題全体のトークン使用量を評価する．

終了制御では，Runaway[11]が終了指示と外部の完了検証を検討している．本稿では，既知の要求文型について API 実行記録から完了条件を検査する．通常継続との比較では，最終成功，API 要求数，総トークン数を評価する．

3. 提案手法

本稿で提案するのは，既存の要約を利用した呼出し・採否制御と，API 実行記録に基づく終了判断の手順である．モデルを再学習せず，最終成功を保つ実行全体のトークン削減を目指す．図 1(a)は履歴要約による次回入力の更新，(b)は API 実行記録に基づく終了判断を示す．この二つの制御は独立に評価する．

3.1 履歴要約の制御手順

行動生成の呼出しを actor，要約専用の呼出しを focus とする．まず，ハーネスは全履歴を保存し，入力長と前回の要約機会終了後の完了ターン数から要約機会を判定する．次に，単回要約の Single と反復要約の Separate は focus で候補を生成し，Inline は actor の行動応答に要約候補を含める．

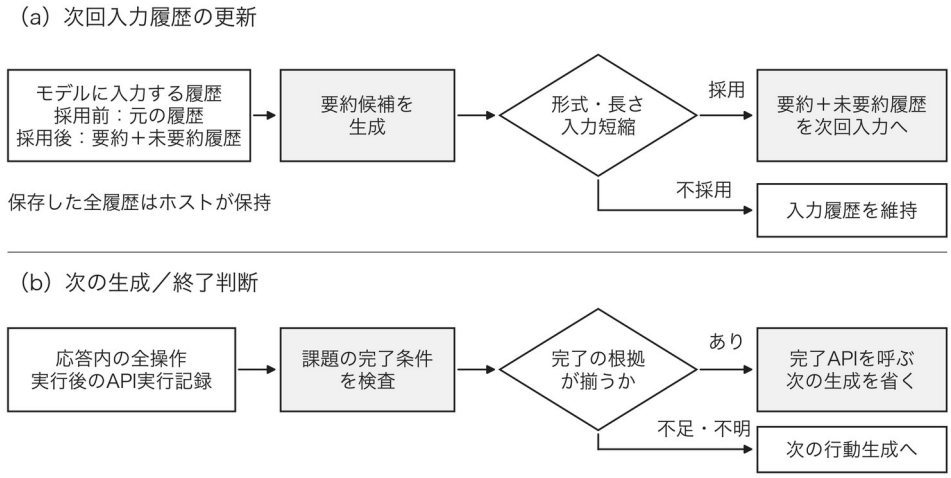


図 1　履歴要約と終了制御

続いて，形式，文字数，次回 actor 入力全体の短縮を検査し，全条件を満たす候補だけ採用する．採用時は元の指示と課題を保持したまま，要約と未要約の履歴を次の actor へ渡し，不採用時は従来の提示履歴を維持する．全履歴は保存し続ける．機会回数と閾値は 4.1 に示し，採否検査は構造と長さを対象とする．

3.2　API 実行記録に基づく終了手順

既知の 1 要求文型に対応する手書きの検査規則を用い，終了を判断する．まず，モデル応答内の全プログラムを実行し，API の要求と応答を記録する．次に，操作前の対象集合と移動前の日時，API から取得した時刻，対象全件の移動先と元の名前の保持を照合する．これらの照合によって完了の根拠が初めて揃えば完了 API を 1 回呼び，次の生成を省く．

文型外の操作や，判定に必要な根拠が不確定・矛盾する場合は通常の次生成へ進む．判定には実行中の API 記録だけを用い，未来応答や公式評価器の正解を参照しない．履歴要約との組合せ効果は評価しない．

3.3　総トークン数と最終成功

条件 p で実行したモデル呼出しの集合を $\mathcal{C}_p$ とする．各呼出し c の入力トークン数を I_c，出力トークン数を O_c とし，実行全体の総トークン数 T_p を式(1)で求める．

$$T_p = \sum_{c \in C_p} \left(I_c + O_c \right) \tag{1}$$

集計には actor と focus の両方を含め，不採用となった要約の生成や失敗時の呼出しについても，取得できた使用量を計上する．使用量が不明な場合は 0 とせず，既知分の小計と不明件数を記録する．

課題の成功は，実行終了後の状態に対する AppWorld[12]の公式評価器で判定する．評価器の正解や詳細は実行中のモデルへ返さない．方式間では成功数と，各課題の成功の維持を確認する．

本指標はキャッシュされた入力も含む論理トークン数であり，金銭課金や実行時間を直接表すものではない[8]．要約生成を含む費用評価は ACON[7]にも用いられている．実験 A と B は独立に集計する．

4.　実験 A：要約の呼出し配置

4.1　条件

写真操作，精算，返金，CSV 処理の既知 4 課題に対し，Single，Separate，Inline の各方式を各課題で 1 回ずつ実行した（4 課題×3 方式，計 12 実行）．全条件は同じ MiniMax-M3 （GGUF，IQ4_XS），llama.cpp b11669，文脈長 65,536 を用いた．モデル生成の seed の基点は 57，環境の seed は 42 で，生成側は呼出しの役割・順序に従い seed を増分した．

要約機会の入力長閾値は 16,384 トークンとし，要約時には直近 2 ターンを原文のまま保持する．反復では，前回の要約機会の終了（採否を問わない）から新たに 3 ターン以上が完了することを要求する．初回の起点は課題開始である．要約機会の上限は Single が 1 回，Separate と Inline が 3 回である．要約は合計 8,192 文字以下とし，次の actor 入力全体が短くなる場合に採用する．Single と Separate の focus 出力上限は 4,096 トークンで，focus の使用量も式(1)へ含める．

各実行の上限はモデル呼出し 80 回，総トークン数 450,000，作業時間 1,800 秒とし，環境 API は写真 160，他 3 課題 80 要求とした．actor 温度 1.0，focus 温

度 0.7，並列実行数 1，KV cache q8_0，計算機は Apple M3 Ultra・メモリ 256 GiB である．実装と全条件の記録を公開する[1]．

4.2 全 4 課題の結果

全 4 課題の成功を保ち，反復要約の Separate は単回要約の Single より総トークン数を 107,873（12.07%）削減した（表 1）．全 12 実行で課題に成功し，使用量の欠測はなく，要約生成を含む課題全体で削減を得た．この差は CSV 処理の 440,461 から 332,588 への減少によるもので，他の 3 課題は同じ値だった．

CSV 処理での要約採用回数は Single が 1 回，Separate と Inline が各 2 回だった．写真操作は全方式で 1 回，精算と返金は 0 回だった．

表 1　実験 A の課題別総トークン数

課題	Single	Separate	Inline
写真操作	139,871	139,871	129,374
精算	211,110	211,110	211,110
返金	102,087	102,087	102,087
CSV 処理	440,461	332,588	394,351
合計	893,529	785,656	836,922

Inline は Single より 6.34%少なかったが，Separate より 51,266 トークン（6.53%）多かった．Separate との比較では，写真操作で 10,497 トークン少ない一方，CSV 処理で 61,763 トークン多かった．要約生成を含む 4 課題の合計では，Separate が 3 方式中で最少となった．図 2 に CSV 処理とその他 3 課題の内訳を示す

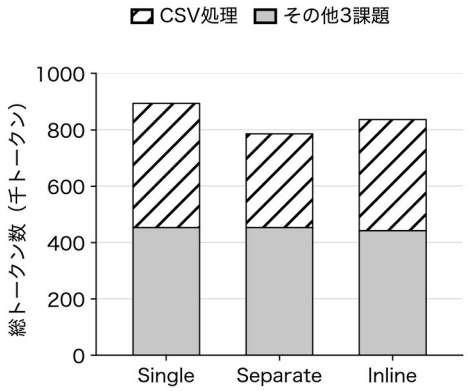


図 2　実験 A の総トークン数

5. 実験 B：API 実行記録に基づく終了

5.1 条件

同じ写真課題について，API 文書の取得を必須とする条件（文書必須）と，公開 API 名からのアクセスを許す条件（名前参照）を設けた．各 API アクセス条件で，通常継続と API 実行記録に基づく終了制御（完了判定）を比較した．完了判定は，事前に実行・保存した通常継続と同じ初期状態，モデル，seed，モデルへの指示とツール説明，予算を用いて各 1 回実行した．

seed の基点は 42 であり，実験 A とは別の履歴・設定である．同じ MiniMax-M3 IQ4_XS と llama.cpp b11669 を用い，各実行の予算はモデル呼出し 80 回，450,000 トークン，1,800 秒，API160 要求とした．

終了判定と完了通知は 3.2 に従う．判定，記録保存，完了通知を含む実行時間を記録し，完了通知を API 要求数に含めた．記録先と集計項目は補足資料 EVIDENCE.md に示す．

5.2 成功を保った総トークン数の削減

両 API 条件で課題の成功を保ち，完了判定により総トークン数を削減した（表 2，図 3）．文書必須では 130,439 から 91,116 へ 30.15%，名前参照では 151,664 から 112,233 へ 26.00%減った．全 4 実行で公式評価器による成功を確認し，使用量の欠測はなかった．各条件でモデル呼出し 2 回と API 要求 4 回を削減した．完了通知を含めて処理を終え，対象の移動後に続いていた 4 回の確認 API と，それに伴う生成を省いた．

表 2　実験 B の総トークン数と呼出し回数

API 条件	終了方法	総トークン数	モデル呼出し数	API 要求数
文書必須	通常継続	130,439	15	104
文書必須	完了判定	91,116	13	100
名前参照	通常継続	151,664	14	103
名前参照	完了判定	112,233	12	99

実行した API 操作などの前半の記録と，実行終了時の 13 個のデータベースのハッシュ値は対照と一致し，最終状態を保った削減を確認した．要求・モデル応答の全文が通常継続側の先頭部分と完全に一致する事前基準は満たさなかったため，結果は既知の要求文型に対する各 1 回の比較として扱う．平均的な因果効果や汎用的な終了保証を示す評価ではない．

1 https://github.com/sophiria1225/AI-

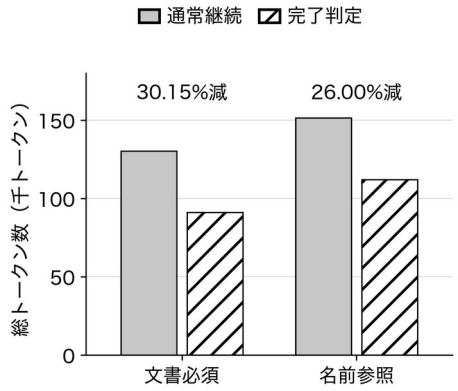


図3　実験Bの総トークン数

6. むすび

既知4課題では，反復要約のSeparateにより全課題の成功を保ち，Singleに比べて総トークン数を12.07%削減した．同じ写真課題の2つのAPI条件では，API実行記録に基づく完了判定により成功を保ち，通常継続に比べて26.00–30.15%削減した．要約生成を含む全モデル呼出しを計測し，両制御を独立に評価して実行費用の削減を確認した．

本稿は，既知課題と単一モデル・量子化設定における少数のseed・試行による開発実験を事後に整理した評価である．未見課題の品質維持や統計的非劣性は評価していない．追加条件の採否理由はEVIDENCE.mdに示した．補足のsupplemental_results.pdfとP144R1_all_conditions.csvには全12実行を収録した．

文　献

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” Proc. ICLR, May 2023.

[2] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better LLM agents,” Proc. ICML, PMLR, vol.235, pp.50208–50232, July 2024.

[3] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” Trans. ACL, vol.12, pp.157–173, 2024. doi: 10.1162/tacl_a_00638.

[4] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing prompts for accelerated inference of large language models,” Proc. EMNLP, pp.13358–13376, Dec. 2023. doi: 10.18653/v1/2023.emnlp-main.825.

[5] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang, “LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression,” Findings of ACL: ACL 2024, pp.963–981, Aug. 2024. doi: 10.18653/v1/2024.findings-acl.57.

[6] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, “LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression,” Proc. ACL, vol.1, pp.1658–1677, Aug. 2024. doi: 10.18653/v1/2024.acl-long.91.

[7] M. Kang, W.-N. Chen, D. Han, H. A. Inan, L. Wutschitz, Y. Chen, R. Sim, and S. Rajmohan, “ACON: Optimizing context compression for long-horizon LLM agents,” Proc. ICML, PMLR, vol.306, pp.55321–55375, July 2026.

[8] B. Xu, Z. Xue, D. Chen, C. Fu, C. Wu, C. Huang, C. Jiang, J. Fang, X. Deng, Y. Chen, Y. Yao, X. Wang, J. Shang, H. Chen, and N. Zhang, “TokenPilot: Cache-efficient context management for LLM agents,” Findings of ACL: EMNLP 2026, pp.19034–19049, Oct. 2026.

[9] J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” Proc. ACM UIST, article 2, pp.1–22, Oct. 2023. doi: 10.1145/3586183.3606763.

[10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” Advances in Neural Information Processing Systems, vol.36, pp.8634–8652, 2023. doi: 10.52202/075280-0377.

[11] Q. Lu, L. Ding, S. Cao, X. Liu, K. Zhang, J. Zhang, and D. Tao, “Runaway is ashamed, but helpful: On the early-exit behavior of large language model-based agents in embodied environments,” Findings of ACL: EMNLP 2025, pp.24014–24027, Nov. 2025. doi: 10.18653/v1/2025.findings-emnlp.1304.

[12] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian, “AppWorld: A controllable world of apps and people for benchmarking interactive coding agents,” Proc. ACL, vol.1, pp.16022–16076, Aug. 2024. doi: 10.18653/v1/2024.acl-long.850.

Abstract
We independently evaluated history summarization and termination control in an LLM-agent harness. Measuring every model call, including summarization, showed that repeated summarization and completion checks based on API execution records reduced total tokens while preserving success on known tasks.
Key words: LLM agents, harness, history summarization, termination control, cost evaluation